# Python –OOP I

# Objective

- OOPs concept
- Class and object
- Instance variable, class variable and methods
- Constructor
- Destructor
- Attributes
- Summary

# OOPs concept

www.object.co.in

# Class and object

OBJECT™ TECHNOLOGIES

## class

- Classes are created by keyword class.
- Attributes are the variables that belong to class.
- Attributes are always public and can be accessed using dot (.) operator. Eg.: Myclass.Myattribute

## object

- State : It is represented by attributes of an object. It also reflects the properties of an object.
- Behavior : It is represented by methods of an object. It also reflects the response of an object with other objects.
- Identity : It gives a unique name to an object and enables one object to interact with other objects.

# Instance, class variable and methods

OBJECT™ TECHNOLOGIES

**Instance Variables**

- Instance variables are variables which are unique to each instance.
- It is defined inside a method and belongs only to the current instance of a class.

**Methods**

- are also called as functions which are defined in a class and describes the behaviour of an object.

**Class variable**

- class variables are variables whose value is assigned in the class.
- is a variable that is shared by all the different objects/instances of a class.

# self

- Class methods must have an extra first parameter in method definition. We do not give a value for this parameter when we call the method, Python provides it.
- If we have a method which takes no arguments, then we still have to have one argument.
- This is similar to **this** pointer in C++ and this reference in Java.
- When we call a method of this object as myobject.method(arg1, arg2), this is automatically converted by Python into MyClass.method(myobject, arg1, arg2) . this is all the special self is about.



www.object.co.in

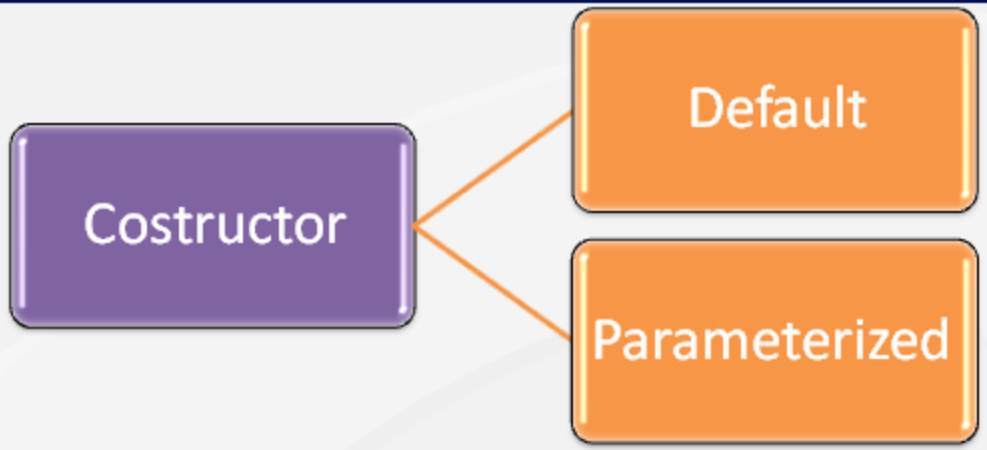
# Example

```
class Parrot:

    # class attribute
    species = "bird"

    # instance attribute
    def __init__(self, name, age):
        self.name = name
        self.age = age

# instantiate the Parrot class
blu = Parrot("Blu", 10)
woo = Parrot("Woo", 15)

# access the class attributes
print("Blu is a {}".format(blu.species))
print("Woo is also a {}".format(woo.species))

# access the instance attributes
print("{} is {} years old".format( blu.name, blu.age))
print("{} is {} years old".format( woo.name, woo.age))
```



www.object.co.in

# Constructors




**Syntax of constructor declaration :**

```
def __init__(self):
    # body of the constructor
```

- Python does not support explicit multiple constructors
- If multiple __init__ methods are written for the same class, then the latest one overwrites all the previous constructors.



www.object.co.in

# Examples of Constructor

OBJECT™ TECHNOLOGIES

```
#Default Constructor
class Student:
    roll_num = 101
    name = "Krushna"

    def display(self):

print(self.roll_num,self.name)

st = Student()
st.display()
```

```
#Parametrized Constructor
class Employee:
    #Parametrised constructor
    def __init__(self, name, id):
        self.id = id
        self.name = name

    def display(self):
        print("ID: %d \nName: %s" % (self.id, self.name))

emp1 = Employee("Gopal",100)
emp2 = Employee("David", 102)

# accessing display() method
emp1.display()
emp2.display()
```

# Destructor

- Destructors are called when an object gets destroyed.
- In Python, destructors are not needed as much needed in C++ because Python has a garbage collector that handles memory management automatically.
- The **__del__()** method is a known as a destructor method in Python.
- It is called when all references to the object have been deleted i.e when an object is garbage collected.

Syntax of destructor declaration :

```
def __del__(self):
        # body of destructor
```



www.object.co.in

# Example

```
# Python program to illustrate destructor
class Employee:

    # Initializing
    def __init__(self):
        print('Employee created.')

    # Deleting (Calling destructor)
    def __del__(self):
        print('Destructor called, Employee deleted.')

obj = Employee()
del obj
```



www.object.co.in

# Attributes

OBJECT™ TECHNOLOGIES

- Attributes in Python defines a property of an object, element or a file.
- There are two types of attributes:

1. **Built-in Class Attributes:**
   - There are various built-in attributes present inside Python classes. For example _dict_, _doc_, _name _, etc.
     E.g. To view all the key-value pairs of employee1.
   - For that, you can simply write the below statement which contains the class namespace:print(emp_1.__dict__)
2. **Attributes defined by Users:**
   - Attributes are created inside the class definition.
   - We can dynamically create new attributes for existing instances of a class.
   - Attributes can be bound to class names as well.



www.object.co.in

# Summary

- OOPs concept
- Class and object
- Instance variable, class variable and methods
- Constructor
- Destructor
- Attributes

# Thank You